

AI Assisted Coding

Lab Assignment – 11.1

Name : P. Gouri Prasad Varma

Hall.t.No. : 2303A51035

Batch : 01

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

1 """
2 Your college wants to develop a Campus Resource Management System
3 that handles:
4 1. Student Attendance Tracking Daily log of students
5 entering/exiting the campus.
6 Implement code using hash table data structure to efficiently store and retrieve attendance records.
7 """
8 class HashTable:
9     def __init__(self, size=100):
10         self.size = size
11         self.table = [None] * self.size
12
13     def hash_function(self, key):
14         return hash(key) % self.size
15
16     def insert(self, key, value):
17         index = self.hash_function(key)
18         if self.table[index] is None:

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
['Alice', 'Bob', 'Charlie']
['David', 'Eve']
['Alice', 'Bob', 'Charlie', 'David']
['Laptop', 'Smartphone']
['Smartphone']
['Smartphone']
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Attendance Tracking	Hash Table	It provides $O(1)$ average time complexity for lookups and updates. Using a Student ID as a key allows for near-instant logging of entry and exit times.
Event Registration	Binary Search Tree (BST)	A BST (or a balanced version like an AVL tree) keeps participant names sorted automatically. This allows for quick alphabetical searches and efficient removals if someone cancels.
Library Borrowing	Hash Table	Searching for a book by its ISBN or unique ID needs to be instantaneous. A Hash Table maps the ID directly to the book's status and due date without scanning the whole collection.
Bus Scheduling	Graph	Routes are essentially a series of nodes (stops) and edges (paths). A Graph allows the system to calculate the shortest path or find all stops connected to a specific route.
Cafeteria Queue	Queue	This follows the FIFO (First-In, First-Out) principle. The first student to line up should be the first one served, ensuring fairness and order in the dining hall.

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor window with a file named `demo1.py`. The code defines a `Queue` class with methods `enqueue` and `dequeue`. The terminal output shows the execution of the script, which prints "Patient A", "Patient B", and "None".

```

45 """
46 A hospital wants to develop a Smart Hospital Management System that handles:
47 1. Patient Check-In System | Patients are registered and treated in order of arrival.
48 Implement code using a queue data structure to manage patient check-ins and ensure that patients are treated in the order they arrived.
49 """
50 class Queue:
51     def __init__(self):
52         self.items = []
53
54     def enqueue(self, item):
55         self.items.append(item)
56
57     def dequeue(self):
58         if not self.is_empty():
59             return self.items.pop(0)
60         return None # Queue is empty
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Patient A
Patient B
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Patient Check-In	Queue	Standard check-ins follow a FIFO (First-In, First-Out) rule. This ensures that non-emergency patients are seen in the exact order they arrived at the desk.
Emergency Handling	Priority Queue	In an ER, the "next" person isn't the one who arrived first, but the one with the highest medical urgency. A Priority Queue allows the system to always serve the most critical case first, regardless of arrival time.
Medical Records	Hash Table	When a doctor needs a patient's history, they need it instantly. Mapping a Patient ID to a record in a Hash Table provides $O(1)$ average time complexity for retrieval.
Appt. Scheduling	BST (or AVL Tree)	Appointments need to be kept in chronological order. A Binary Search Tree keeps them sorted by time, making it easy to find "the next available slot" or print a day's schedule in order.
Room Navigation	Graph	A hospital is a network of wards, floors, and corridors. A Graph represents these locations as nodes and the hallways as edges, allowing for "shortest path" navigation (e.g., from the ER to the ICU).

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor with a file named `demo1.py` open. The code defines a `TrafficQueue` class with methods `enqueue` and `dequeue`. The `dequeue` method uses `pop(0)` to remove the first element from the `vehicles` list. Below the code, a terminal window shows the command `python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"` being executed, with the output: `Vehicle 1`, `Vehicle 2`, and `None`.

```

75 """
76 A city plans a Smart Traffic Management System that includes:
77 1. Traffic Signal Queue Vehicles waiting at signals
78 Implement code using a queue data structure to manage vehicles waiting at traffic signals, ensuring that vehicles are processed in
79 the order they arrived.
80 """
81 class TrafficQueue:
82     def __init__(self):
83         self.vehicles = []
84     def enqueue(self, vehicle):
85         self.vehicles.append(vehicle)
86     def dequeue(self):
87         if not self.is_empty():
88             return self.vehicles.pop(0)
89         return None # Queue is empty
90

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Vehicle 1
Vehicle 2
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Traffic Signal Queue	Queue	This follows the FIFO (First-In, First-Out) principle. The first vehicle to reach the red light should be the first to move when it turns green to maintain a fair and natural traffic flow.
Emergency Priority	Priority Queue	Ambulances and fire trucks cannot wait in a standard line. A Priority Queue ensures these vehicles are bumped to the front of the "processing" list regardless of when they arrived at the intersection.
Vehicle Lookup	Hash Table	Police or automated cameras need to link a license plate to a driver instantly. A Hash Table provides $O(1)$ average time complexity, allowing for near-instant retrieval among millions of records.
Road Network	Graph	A city is a collection of intersections (nodes) and roads (edges). A Graph is the only way to logically map these connections and run algorithms like Dijkstra's to find the fastest route across town.
Parking Slots	Hash Table	To track whether slot "A-101" is occupied, a Hash Table (or a simple Bit Map/Array if IDs are contiguous) allows for instant

Feature	Data Structure	Justification
		status checks and updates without searching through the whole lot.

Task Description #12: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor with a file named `demo1.py`. The code defines a `Node` class and a `ShoppingCart` class. The `Node` class has an `__init__` method that takes a `product` and sets `self.product` and `self.next` to `None`. The `ShoppingCart` class has an `__init__` method that sets `self.head` to `None` and an `add_product` method that creates a new `Node` and adds it to the `head` of the list. The terminal output shows the execution of the script, which prints the list of products: `['Laptop', 'Smartphone']`.

```

105 """
106 An e-commerce company wants to build a Smart Online Shopping System with:
107 1. Shopping Cart Management Add and remove products dynamically.
108 Implement code using a linked list data structure to manage the shopping cart, allowing customers to add and remove products
109 efficiently.
110 """
111 class Node:
112     def __init__(self, product):
113         self.product = product
114         self.next = None
115 class ShoppingCart:
116     def __init__(self):
117         self.head = None
118     def add_product(self, product):
119         new_node = Node(product)
120         if self.head is None:
121             self.head = new_node

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
['Laptop', 'Smartphone']
['Smartphone']
['Smartphone']
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Shopping Cart	Linked List	A Doubly Linked List allows for $O(1)$ additions and removals of products at any point. This is ideal for a dynamic cart where users might remove an item from the middle of their list without re-indexing the entire collection.
Order Processing	Queue	Orders must be handled using the FIFO (First-In, First-Out) principle to ensure fairness. The first customer to click "Checkout" is the first whose order is sent to the warehouse for packing.
Top-Selling Tracker	Priority Queue	A Max-Priority Queue (often implemented as a Heap) keeps the products with the highest sales counts at the top. This allows the platform to instantly retrieve the "Top N" trending products for the homepage.
Search Engine	Hash Table	When a user enters a specific Product ID or SKU, the system needs to find that product instantly among millions. A Hash Table provides $O(1)$ average time complexity for these direct lookups.
Delivery Routes	Graph	Warehouses and delivery addresses are nodes, while roads are edges. A Graph structure is necessary to calculate the most

Feature	Data Structure	Justification
		efficient path for drivers, often using algorithms like Dijkstra's or A*.

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor window with a file named `demo1.py`. The code defines a `Stack` class with methods `__init__`, `push`, and `pop`. The `pop` method checks if the stack is empty before popping. Below the code, the terminal shows the execution of the script, which outputs "Shipment B", "Shipment A", and "None".

```

161
162
163 """
164 1. Shipment Processing
165 Implement code using a stack data structure to manage shipment processing, allowing the company to process shipments in a last-in-first
166 """
167 class Stack:
168     def __init__(self):
169         self.shipments = []
170
171     def push(self, shipment):
172         self.shipments.append(shipment)
173
174     def pop(self):
175         if not self.is_empty():
176             return self.shipments.pop()
177         return None # Stack is empty

```

```

PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Shipment B
Shipment A
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>

```

Feature	Data Structure	Justification
Shipment Processing	Queue	Standard shipments are processed using the FIFO (First-In, First-Out) principle. This ensures that the first package to arrive at the sorting facility is the first one loaded onto a truck.
Urgent Shipment Priority	Priority Queue	"Express" or "Overnight" packages must jump the line. A Priority Queue allows the system to always extract the highest-priority item next, regardless of when it entered the system.
Product Lookup by Code	Hash Table	Warehouses contain millions of SKUs. A Hash Table allows a worker to scan a barcode and retrieve the product's aisle, shelf, and quantity in $O(1)$ average time.
Warehouse Network	Graph	Logistics hubs and distribution centers are nodes connected by transport routes (edges). A Graph is essential for calculating the most cost-effective path between two geographical points.
Inventory Operations	Linked List	For a dynamic inventory where products are frequently inserted or removed from various points in a list (not just the ends), a Linked List offers flexible $O(1)$ pointer updates without shifting elements.

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a Visual Studio Code editor window with a file named `demo1.py`. The code defines a `CustomerQueue` class with methods for enqueueing and dequeuing customers. The terminal output shows the execution of the script, which prints "Customer 1", "Customer 2", and "None".

```
191
192 """
193 A bank plans to develop a Smart Banking System that includes:
194 1. Customer Service Token System | Customers are served in order of arrival.
195 Implement code using a queue data structure to manage customer service tokens, ensuring that customers are served in the order they arrive.
196 """
197 class CustomerQueue:
198     def __init__(self):
199         self.customers = []
200
201     def enqueue(self, customer):
202         self.customers.append(customer)
203
204     def dequeue(self):
205         if not self.is_empty():
206             return self.customers.pop(0)
207         return None # Queue is empty
208
209
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Customer 1
Customer 2
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Feature	Data Structure	Justification
Customer Token System	Queue	This follows the FIFO (First-In, First-Out) principle. It ensures fair treatment by serving the customer who took the first token before those who arrived later.
Loan Approval Priority	Priority Queue	Not all loans have the same urgency; a high-value corporate loan or an emergency credit line may need faster processing. A Priority Queue allows the bank to "jump" these cases to the front of the review pile.
Account Lookup	Hash Table	A bank manages millions of accounts. Using the Account Number as a key in a Hash Table allows for $O(1)$ average time complexity, retrieving details instantly without searching the entire database.
Transaction History	Linked List	Transactions are sequential and constantly growing. A Linked List (specifically a Doubly Linked List) allows the bank to easily append new transactions or traverse through history chronologically without needing contiguous memory.
ATM Network	Graph	ATMs are physical locations (nodes) connected by maintenance routes or data links (edges). A Graph helps the bank optimize

Feature	Data Structure	Justification
		cash replenishment routes or analyze the connectivity between different city branches.

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

```
File Edit Selection View Go ... Q Siddu (Workspace) demo1.py X
Programs > Python > AIAC > demo1.py > ...
220
221 """
222 An online education platform wants to develop a Learning Management System that handles:
223 1. Student Login Authentication Quick validation of user credentials.
224 Implement code using a hash table data structure to efficiently store and retrieve student login credentials for authentication purposes.
225 """
226 class HashTable:
227     def __init__(self, size=100):
228         self.size = size
229         self.table = [None] * self.size
230
231     def hash_function(self, key):
232         return hash(key) % self.size
233
234     def insert(self, key, value):
235         index = self.hash_function(key)
236         if self.table[index] is None:
```

```
PROBLEMS PORTS OUTPUT DEBUG CONSOLE TERMINAL
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
password123
securepass
newpassword
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Ln 259, Col 63 Spaces: 4 UTF-8 CRLF Python 3.13.7 Go Live 14:43 12-03-2026

Feature	Data Structure	Justification
Login Authentication	Hash Table	Student credentials (usernames/hashed passwords) need to be validated instantly. A Hash Table provides $O(1)$ average time complexity, ensuring that login speed remains constant even as the student body grows.
Assignment Submission	Queue	This follows the FIFO (First-In, First-Out) principle. It ensures that the grading system or the storage processor handles assignments in the exact order they were uploaded by students.
Top Performer Ranking	Priority Queue	Ranking students by scores is most efficient with a Max-Heap. A Priority Queue allows the platform to instantly extract the highest-scoring students for a leaderboard without sorting the entire database.
Course Prerequisites	Graph	Courses and their dependencies form a complex network. A Directed Acyclic Graph (DAG) is perfect for representing that Course A must be completed before Course B, allowing for topological sorting.
Undo Feature (Editor)	Stack	This relies on the LIFO (Last-In, First-Out) principle. Every action is "pushed" onto the stack; when a student hits

Feature	Data Structure	Justification
		"Undo," the most recent action is "popped" off to revert the state.

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

The screenshot shows a VS Code editor window with a file named `demo1.py`. The code defines a `PassengerQueue` class with methods for enqueueing and dequeueing passengers. The terminal output shows the script being executed, resulting in the output: `Passenger A`, `Passenger B`, and `None`.

```
265 """
266 An airport authority plans to implement:
267 1. Passenger Check-In Queue | Process passengers in order of arrival.
268 Implement code using a queue data structure to manage passenger check-ins, ensuring that passengers are processed in the order they
269 arrived.
270 """
271 class PassengerQueue:
272     def __init__(self):
273         self.passengers = []
274
275     def enqueue(self, passenger):
276         self.passengers.append(passenger)
277
278     def dequeue(self):
279         if not self.is_empty():
280             return self.passengers.pop(0)
281         return None # Queue is empty
282 """
```

```
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Passenger A
Passenger B
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Feature	Data Structure	Justification
Passenger Check-In	Queue	Check-in counters operate on a FIFO (First-In, First-Out) basis. This ensures that the first passenger to join the line is the first one assisted, maintaining order and fairness during peak travel times.
Emergency Landing	Priority Queue	In aviation, safety overrides arrival order. A Priority Queue allows the Air Traffic Control system to automatically move aircraft with emergency codes (e.g., low fuel, engine failure) to the top of the landing list.
Flight Lookup	Hash Table	Travelers and staff need to find flight status instantly among hundreds of daily departures. A Hash Table using the Flight ID as a key provides $O(1)$ average time complexity for these searches.
Air Route Connectivity	Graph	Global aviation is a giant network. Airports are nodes and flight paths are edges; a Graph is necessary to calculate connecting flights, total travel distance, and fuel requirements.
Boarding (Luggage)	Stack	Overhead bins and narrow cabin aisles often function on a LIFO (Last-In, First-Out) basis. The last bag pushed into a

Feature	Data Structure	Justification
		deep storage compartment or the last person in a narrow jet bridge is often the first one to come out.

Task Description #17: Smart Social Media Platform – Data Structure

Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments

```

294
295 """
296 A social media company wants to develop a system that includes:
297 1. News Feed Generation [ ] Display posts chronologically.
298 Implement code using a linked list data structure to manage the news feed, allowing for efficient insertion of new posts and
299 retrieval of posts in chronological order.
300 """
301 class PostNode:
302     def __init__(self, post):
303         self.post = post
304         self.next = None
305 class NewsFeed:
306     def __init__(self):
307         self.head = None
308
309     def add_post(self, post):
310         new_node = PostNode(post)
311         if self.head is None:
312             self.head = new_node
313         else:
314             current = self.head
315             while current.next:
316                 current = current.next
317             current.next = new_node
318
319     def get_news_feed(self):
320         posts = []
321         current = self.head
322         while current:

```

Feature	Data Structure	Justification
News Feed	Linked List	A Doubly Linked List is excellent for chronologically ordered data. It allows for $O(1)$ insertions at the head (newest post) and efficient traversal to load "older" posts as the user scrolls down.
Trending Hashtags	Priority Queue	To find what's "trending," the system needs to constantly track the most frequently used hashtags. A Max-Heap (Priority Queue) allows the top N hashtags to be extracted instantly without sorting the entire dataset.
Profile Search	Hash Table	Username are unique identifiers. Mapping a username to a profile object in a Hash Table ensures that searching for a friend happens in $O(1)$ average time, regardless of how many millions of users exist.
Friend Network	Graph	Social connections are non-linear and multidimensional. A Graph (using an Adjacency List) perfectly represents users as nodes and friendships as edges, enabling features like "Suggested Friends" or "Degrees of Separation."
Message Undo	Stack	This follows the LIFO (Last-In, First-Out) principle. By pushing sent messages onto a Stack, the system can easily "pop" the most

Feature	Data Structure	Justification
		recent message to remove it if the user hits the undo/withdraw button.

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings

```

File Edit Selection View Go ...
Q: Siddu (Workspace)
demo1.py x
Programs > Python > AIAC > demo1.py > ...
331
332
333
334 """
335 A university wants to build a hostel management system that manages:
336 1. Room Allotment
337 Implement code using a hash table data structure to efficiently store and retrieve room allotment information for students.
338 """
339 class HashTable:
340     def __init__(self, size=100):
341         self.size = size
342         self.table = [None] * self.size
343
344     def hash_function(self, key):
345         return hash(key) % self.size
346
347     def insert(self, key, value):
348         index = self.hash_function(key)
349         if self.table[index] is None:
350             self.table[index] = [(key, value)]
351         else:
352             for i, (k, v) in enumerate(self.table[index]):
353                 if k == key:
354                     self.table[index][i] = (key, value) # Update existing key
355             return
356             self.table[index].append((key, value)) # Add new key
357
358     def retrieve(self, key):
359         index = self.hash_function(key)
360
361
Ln 370, Col 56 Spaces: 4 UTF-8 CRLF Python 3.13.7 Go Live
Indexing completed.

```

Feature	Data Structure	Justification
Room Allotment	Queue	Rooms are generally allotted on a "First-Come, First-Served" basis during the registration period. A Queue ensures that the student who applied first gets the first choice of available rooms.
Medical Emergency	Priority Queue	In a hostel, health crises require immediate attention. A Priority Queue allows the system to process medical alerts based on severity (e.g., an unconscious student vs. a minor fever) rather than arrival time.
Student Record Retrieval	Hash Table	Retrieving a student's profile, contact info, or fee status by their Roll Number must be near-instant. A Hash Table provides $O(1)$ average time complexity, making lookups efficient regardless of hostel size.
Hostel Block Connectivity	Graph	A large campus has multiple blocks, messes, and security gates connected by pathways. A Graph represents these locations as nodes and paths as edges, helping in planning security patrols or maintenance routes.
Visitor Entry Log	Linked List	Visitors are logged sequentially as they enter and leave. A Linked List (or a Doubly Linked List) allows for efficient

Feature	Data Structure	Justification
		appending of new entries and is flexible for time-based tracking without requiring a fixed memory block.

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

The screenshot shows a VS Code editor window with a file named `demo1.py`. The code defines a `ComplaintQueue` class with methods for enqueueing and dequeuing complaints. The terminal output shows the execution of the script, which prints out two complaints: 'Complaint 1: Power outage in area A' and 'Complaint 2: Billing issue for customer B', followed by 'None'.

```
375 """
376 An electricity board plans to develop a system that includes:
377 1. Complaint Handling
378 Implement code using a queue data structure to manage customer complaints, ensuring that complaints are addressed in the order they
379 were received.
380 """
381 class ComplaintQueue:
382     def __init__(self):
383         self.complaints = []
384     def enqueue(self, complaint):
385         self.complaints.append(complaint)
386     def dequeue(self):
387         if not self.is_empty():
388             return self.complaints.pop(0)
389         return None # Queue is empty
390
```

```
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo1.py"
Complaint 1: Power outage in area A
Complaint 2: Billing issue for customer B
None
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Feature	Data Structure	Justification
Complaint Handling	Queue	General complaints (like billing queries) should be addressed in the order they were received. A Queue follows the FIFO (First-In, First-Out) principle, ensuring fair and organized customer service.
Fault Priority Handling	Priority Queue	Not all power outages are equal; a total blackout at a hospital takes precedence over a flickering streetlamp. A Priority Queue allows the system to dispatch repair crews to critical faults first.
Consumer Record Lookup	Hash Table	With millions of consumers, finding details by a Consumer ID must be instantaneous. A Hash Table provides $O(1)$ average time complexity for retrieval, making it the fastest choice for direct lookups.
Power Grid Connectivity	Graph	The grid consists of power plants, substations, and transformers (nodes) connected by transmission lines (edges). A Graph is necessary to model flow, identify points of failure, and reroute power during maintenance.
Meter Reading History	Linked List	Meter readings are sequential data points recorded over time. A Linked List (or a Doubly Linked List) allows for efficient

Feature	Data Structure	Justification
		appending of new monthly readings and chronological traversal of usage history.